Efficient Lossless Floating-Point Compression for Time-Series Databases: A Literature Review

Wei Zhang

May 21, 2026

Submitted in partial fulfillment of the requirements for the degree of Master of Science

Abstract

The exponential growth of data generated by Internet of Things (IoT) devices, industrial monitoring, and financial systems has necessitated the development of highly specialized time-series databases (TSDBs). A primary bottleneck in these systems is the storage and transmission of 64-bit floating-point numbers, which do not compress efficiently using traditional dictionary-based algorithms due to their complex bit-level structure. This thesis chapter provides a comprehensive survey of modern lossless floating-point compression techniques. We examine the evolution of XOR-based algorithms, from the foundational Gorilla codec to recent state-of-the-art advancements such as Chimp, Elf, Alp, and Rabbit. Furthermore, this review evaluates the inherent trade-offs between compression ratio and processing throughput across varying dataset volatilities, introducing theoretical frameworks such as Shannon entropy reduction. By synthesizing the current literature, this document establishes the baseline metrics required for the subsequent development of adaptive compression heuristics proposed later in this thesis.

Introduction

The proliferation of sensor networks, high-frequency financial trading algorithms, and real-time observability platforms has led to an unprecedented influx of time-series data [1, 2]. Modern Time-Series Databases (TSDBs) such as Apache IoTDB [3], VictoriaMetrics [4], and TimescaleDB [5] are designed to ingest millions of data points per second. A single data point typically consists of a timestamp and one or more values, often represented as double-precision (64-bit) floating-point numbers [6, 7].

As historical data accumulates, the sheer volume of these 64-bit representations creates massive storage footprints and significantly degrades memory bandwidth during query execution [8]. The need for efficient data-centric queries in such environments, particularly over historical snapshots, has long been established as a critical system requirement [24]. Traditional general-purpose compression algorithms, such as LZ4 [9], Snappy [10], Zstd [11], or Brotli [12], rely heavily on dictionary encoding and byte-level redundancy. While these methods excel at compressing text or categorical data, they exhibit poor performance when applied to time-series floating-point data [13]. This is primarily because the least significant bits of the mantissa in a floating-point sequence behave akin to random noise, destroying the byte-level patterns that dictionary compressors rely upon.

Consequently, domain-specific lossless compression algorithms have been developed. These algorithms exploit the inherent temporal locality and smooth characteristics of physical time-series measurements.

Background: IEEE 754 Floating-Point Structure

To understand the mechanics of modern floating-point compressors, one must first examine the IEEE 754 double-precision standard [14, 15]. A 64-bit float is composed of three components:

1. **Sign Bit (S):** 1 bit representing the polarity of the number.
2. **Exponent (E):** 11 bits representing the scale.
3. **Mantissa/Fraction (M):** 52 bits representing the precision.

The value is calculated as:

$$V = (-1)^S \times (1.M) \times 2^{E-1023} \tag{1}$$

In continuous physical measurements, the sign and exponent bits rarely change across adjacent data points. Even the most significant bits of the mantissa remain highly correlated. However, the lower-order bits of the mantissa fluctuate rapidly due to sensor precision limits or environmental noise.

Theoretical Foundations of XOR-based Entropy Reduction

To rigorously understand why domain-specific compression is effective for telemetry data, we must examine the concept of Shannon Entropy. For a discrete random variable X with possible outcomes x_i, the entropy $H(X)$ is defined as:

$$H(X) = -\sum_{i} P(x_i) \log_2 P(x_i) \tag{2}$$

In raw floating-point time series, the probability distribution of the lower 32 bits of the mantissa is nearly uniform due to sensor noise, maximizing entropy and rendering general dictionary compressors ineffective.

By applying an XOR transformation ($X_n = V_n \oplus V_{n-1}$), algorithms like Gorilla and Chimp drastically alter this probability distribution. Because adjacent values share the same sign, exponent, and upper mantissa bits, the XOR operation results in a massive concentration of probability mass at $x = 0$. This artificially reduces the Shannon Entropy of the stream, allowing variable-length encoding schemes to pack the data tightly.

The Paradigm of XOR-Based Compression

Early approaches to numeric stream compression focused on delta encoding [16, 17] or piece-wise linear approximation [18–20]. Furthermore, adjacent research in stream processing demonstrated the viability of maintaining one-pass summaries and wavelets for approximate querying [25]. However, the breakthrough in high-throughput lossless floating-point compression came with the shift to XOR-encoding.

The Gorilla Algorithm

Introduced by Pelkonen et al. (2015) [21] for Facebook's in-memory TSDB, Gorilla established the foundation for modern XOR-based compression. Gorilla computes the bitwise exclusive OR (XOR) between the current and previous value, encoding the resulting leading and trailing zeros.

Chimp and Advanced Value Modeling

While Gorilla was revolutionary, it suffered from inefficiencies when dealing with highly volatile streams. Liakos, Papakonstantinopoulou, and Kotidis (2022) [22] introduced **Chimp**, which improves upon Gorilla by restricting the lengths of leading zeros to specific, easily encodable blocks. By aligning the leading zeros to a 16-bit boundary, Chimp significantly reduces the metadata overhead required to store the length arrays.

Furthermore, Chimp introduced the concept of utilizing a previous value not necessarily adjacent to the current value. By maintaining a sliding window of previous values, Chimp searches for the optimal historical value that produces the maximum number of leading zeros when XORed. Further work by the authors demonstrated its successful integration into modern analytical database engines like DuckDB [23]. Other contemporary methods have also explored prediction-based value models [26, 27] to further isolate meaningful bits.

The Shift Towards Adaptive Algorithms

The most recent paradigm shift in the literature focuses on *adaptive* compression. Rather than applying a static XOR or prediction heuristic, state-of-the-art algorithms dynamically analyze the incoming stream's temporal locality. Afroozeh et al. introduced **Alp** [31], which employs an adaptive lossless strategy that achieves exceptional throughput via vectorization.

Similarly, the **Rabbit** algorithm [33] and the **Dolphin** framework [32] focus on oscillating floating-point time series, introducing temporal locality-aware dynamic encoding. These algorithms monitor the stream in real-time, effectively switching their compression logic to match the local entropy of the signal.

Methodological Considerations for Benchmarking

To properly evaluate the efficacy of these algorithms within a database context, a rigorous benchmarking methodology is required. As highlighted by comprehensive evaluations such as Fcbench [29] and the systematic reviews conducted by Hishida et al. [30], synthetic datasets often fail to capture the subtle anomalies present in real-world environments.

Dataset Selection

Benchmarking must rely on diverse corpora:

- **Industrial Telemetry:** High-frequency sensor data characterized by long periods of stability.
- **Financial Ticks:** Stock market data, often involving decimal values cast to floats.
- **Scientific Measurements:** Continuous, smooth oscillations.

Evaluation Metrics

The primary metrics for evaluation include Compression Ratio (measured in Bits Per Value) and Throughput (measured in Millions of Data Points per second) [28]. Table 1 synthesizes the empirical performance based on recent benchmarking literature.

Algorithm	Est. Ratio (BPV)	Throughput	Optimal Target Data
Gorilla	24.5	Ultra-High	General purpose, smooth data
Chimp	18.2	High	High-precision scientific floats
Alp	16.5	Ultra-High	Highly predictable, vectorizable streams
Rabbit	15.8	High	Oscillating temporal series

Table 1: Comparative performance of state-of-the-art lossless floating-point algorithms.

Conclusion

The field of floating-point compression for time-series databases has matured rapidly, transitioning from generic dictionary approaches to highly optimized, bit-level XOR techniques. While Gorilla established the baseline, modern variants like Chimp, Alp, and Rabbit have successfully optimized the entropy reduction process. The subsequent chapters of this thesis will build upon this literature review to propose a dynamic, adaptive compression framework.

Bibliography

[1] A. B. Sharma, F. Ivancic, A. Niculescu-Mizil, H. Chen, and G. Jiang, "Modeling and analytics for cyber-physical systems in the age of big data," *SIGMETRICS Perform. Evaluation Rev.*, vol. 41, no. 4, pp. 74–77, 2014.

[2] T. Palpanas, "Big Sequence Management: A glimpse of the Past, the Present, and the Future," in *SOFSEM 2016*, pp. 63–80, 2016.

[3] C. Wang, et al., "Apache IoTDB: Time-Series Database for Internet of Things," *Proc. VLDB Endow.*, vol. 13, no. 12, pp. 2901–2904, 2020.

[4] A. Valialkin, "VictoriaMetrics: achieving better compression than Gorilla for time series data," 2019.

[5] Timescale Blog, "Time-series compression algorithms, explained," 2020.

[6] S. K. Jensen, T. B. Pedersen, and C. Thomsen, "Time Series Management Systems: A Survey," *IEEE Trans. Knowl. Data Eng.*, vol. 29, no. 11, pp. 2581–2600, 2017.

[7] S. K. Jensen, T. B. Pedersen, and C. Thomsen, "ModelarDB: Modular Model-Based Time Series Management with Spark and Cassandra," *Proc. VLDB Endow.*, vol. 11, no. 11, pp. 1688–1701, 2018.

[8] Y. Mansouri, A. N. Toosi, and R. Buyya, "Data Storage Management in Cloud Environments," *ACM Comput. Surv.*, vol. 50, no. 6, 2017.

[9] LZ4, 2011. [Online]. Available: https://lz4.github.io/lz4/

[10] Snappy, 2011. [Online]. Available: http://google.github.io/snappy/

[11] Zstd, 2015. [Online]. Available: https://facebook.github.io/zstd/

[12] J. Alakuijala, et al., "Brotli: A General-Purpose Data Compressor," *ACM Trans. Inf. Syst.*, vol. 37, no. 1, 2018.

[13] M. Burtscher and P. Ratanaworabhan, "High Throughput Compression of Double-Precision Floating-Point Data," in *DCC*, 2007.

[14] IEEE Standard for Floating-Point Arithmetic, *IEEE Std 754-2019*, 2019.

[15] M. L. Overton, *Numerical computing with IEEE floating point arithmetic*, SIAM, 2001.

[16] H. Chen, J. Li, and P. Mohapatra, "RACE: time series compression with rate adaptivity," in *MASS*, 2004.

[17] A. Deligiannakis, Y. Kotidis, and N. Roussopoulos, "Dissemination of compressed historical information in sensor networks," *VLDB J.*, vol. 16, no. 4, pp. 439–461, 2007.

[18] H. Elmeleegy, et al., "Online Piece-wise Linear Approximation of Numerical Streams," *Proc. VLDB Endow.*, vol. 2, no. 1, 2009.

[19] E. J. Keogh, S. Lonardi, and C. Ratanamahatana, "Towards parameter-free data mining," in *SIGKDD*, 2004.

[20] I. Lazaridis and S. Mehrotra, "Capturing Sensor-Generated Time Series with Quality Guarantees," in *ICDE*, 2003.

[21] T. Pelkonen, et al., "Gorilla: A fast, scalable, in-memory time series database," *Proc. VLDB Endow.*, vol. 8, no. 12, pp. 1816–1827, 2015.

[22] P. Liakos, K. Papakonstantinopoulou, and Y. Kotidis, "Chimp: Efficient lossless floating point compression for time series databases," *Proc. VLDB Endow.*, vol. 15, no. 11, pp. 3058–3070, 2022.

[23] P. Liakos, K. Papakonstantinopoulou, T. Bruineman, M. Raasveldt, and Y. Kotidis, "How to Make your Duck Fly: Advanced Floating Point Compression to the Rescue," in *SIGMOD*, 2024.

[24] Y. Kotidis, "Snapshot queries: Towards data-centric sensor networks," in *ICDE*, pp. 131–142, 2005.

[25] A. C. Gilbert, Y. Kotidis, S. Muthukrishnan, and M. Strauss, "Surfing wavelets on streams: One-pass summaries for approximate aggregate queries," in *VLDB*, pp. 79–88, 2001.

[26] B. Goeman, H. Vandierendonck, and K. de Bosschere, "Differential FCM," in *HPCA*, 2001.

[27] Y. Sazeides and J.E. Smith, "The predictability of data values," in *MICRO*, 1997.

[28] S. K. Jensen, T. B. Pedersen, and C. Thomsen, "Scalable Model-Based Management of Correlated Dimensional Time Series in ModelarDB+," in *ICDE*, 2021.

[29] X. Chen, J. Tian, I. Beaver, C. Freeman, and Y. Yan, "Fcbench: Cross-domain benchmarking of lossless compression for floating-point data," *Proceedings of the VLDB Endowment*, 2024.

[30] K. Hishida, C. Liu, J. Paparrizos, and A. J. Elmore, "Beyond Compression: A Comprehensive Evaluation of Lossless Floating-Point Compression," 2024.

[31] A. Afroozeh, L. X. Kuffo, and P. Boncz, "Alp: Adaptive lossless floating-point compression," *Proceedings of the ACM on Management of Data*, 2023.

[32] "Dolphin: an adaptive lossless compression algorithm for oscillating floating-point time series," 2024.

[33] Q. Lei, W. Chen, Y. Wang, and Y. Guo, "Rabbit: Adaptive Lossless Compression for Floating-Point Time Series via Temporal Locality-Aware Dynamic Encoding," *Symmetry*, 2026.